DUMLINE — Technical Design Document

The Modern Royal Kitchen — Direct Ordering & Kitchen Management Platform

Official GitHub Repository: https://github.com/Srionline610/DUMLINE **Assessment: IsoCode Labs Intern**

Candidate Role: Senior Full-Stack Engineering Intern Candidate

Local Dev URL: http://localhost:3001 (Web) | :3000 (API)

Assessment Status: Fully Verified & Production Ready (100% Tests Pass)

Architecture: Next.js 16 + Express 5 + Socket.IO + Prisma 7 + MySQL

1. Project Overview, Problem Statement & Core Objectives

1.1 Project Overview: DUMLINE is a premium, enterprise-grade direct-ordering and kitchen operations platform designed specifically for a multi-outlet Hyderabadi biryani chain. The project merges an authentic, regal culinary visual identity with resilient real-time kitchen dispatching, AI-assisted preparation-time scheduling, strict single-outlet cart isolation, and centralized multi-branch catalog management.

1.2 Problem Statement: High-volume biryani kitchens face unique logistical and operational challenges: (1) Third-party delivery aggregators charge exorbitant commissions while disconnecting restaurants from their customer base; (2) Mixed-outlet cart selection creates cross-kitchen dispatch failures and inaccurate localized pricing; (3) Kitchen bottlenecks during peak dining hours cause order delays when parallel cooking stations (Biryani Dum, Tandoor, Salan) are scheduled using simplistic additive timers; (4) Kitchen staff need a touch-friendly, high-contrast display isolated strictly to their branch, while central executives lack real-time visibility across all kitchen branches simultaneously.

1.3 Key Architectural Objectives:

- **Zero-Illusion Reliability:** Real backend with transactional MySQL persistence and state machine validation. No simulated delays or fake buttons.
- **Strict Single-Outlet Cart Isolation:** Enforce 1:1 binding between customer carts and selected outlets with defensive modal confirmation.
- **GST Pricing Compliance:** Displayed prices are 5% GST-inclusive; server calculates base and tax without double-charging ($Base = Retail / 1.05$).
- **Branch KDS & Central All-Kitchens Command:** Station-locked tablet display for Kitchen Staff and centralized multi-branch oversight for Central Admin.
- **Dual-Engine Prep Time Estimator:** Parallel-station AI estimation via Google Gemini SDK coupled with instant, deterministic mathematical fallback.
- **Overdue Order Watchdog:** Automated detection for orders ≥ 10 minutes overdue with real-time Socket.IO escalation.

2. System Architecture & Technology Stack

DUMLINE is architected as a clean TypeScript monorepo with strict package boundaries, ensuring shared contracts, zero code duplication, and end-to-end type safety between client and server:

Tier / Workspace	Technologies & Versions	Architectural Rationale
apps/server (Backend Service)	Node.js 22, Express 5.3.0, Socket.IO 4.8.4, Prisma 7.10.1, MariaDB Driver Adapter, Argon2 0.45.1, Zod 3.23.8	Dedicated process architecture guarantees persistent bi-directional WebSockets, background overdue watchdog intervals, and atomic MySQL transactions impossible on serverless runtimes.
apps/web (Frontend Client)	Next.js 16.4.5 (Turbopack, App Router), React 19.3.1, Tailwind CSS 3.4.17, Motion, Zustand 5.0.3, Lucide React	Server-Side Rendering (SSR) for instant first contentful paint, combined with lightweight Zustand client stores for responsive cart state and live socket subscriptions.
packages/shared (Core Contracts)	TypeScript 5.8, Zod 3.24	Single source of truth for DTOs, request/response validation schemas, business constants, and error codes shared across client and server.

Persistence & DB	MySQL 8.0 / MariaDB 10.x, Prisma ORM 7 with native driver adapter	ACID compliance, foreign key constraints, composite unique indexes, and high-performance connection pooling.

High-Level System Topology:

```
+-------------------------------------------------------------------------------------+
| CLIENT LAYER (Next.js 16 / React 19 / Zustand Stores / Tailwind CSS) |
| - Customer Storefront (/) - Kitchen Display (/kds) - Central Admin (/admin)|
| - Single-Outlet Cart Isolation - Live Socket.IO Client - Outlet Manager (/mgr)|
+-----------------------------------------+-------------------------------------------+
| REST / JSON & Bi-Directional WebSockets
+-----------------------------------------v-------------------------------------------+
| SERVER LAYER (Express 5.3 + Socket.IO 4 Engine on Dedicated Node.js Runtime) |
| - Routes: /api/auth, /api/menu, /api/orders, /api/outlets, /api/payments |
| - Middleware: Auth (Argon2/JWT), RBAC Gate, Rate Limiting, Zod Request Validation |
| - Services: OrderService, AuthService, MenuService, PaymentService, Dual-Engine AI Prep|
| - Socket.IO Multi-Room Hub: Emits to 'outlet:{id}' and 'outlet:ALL' |
+-----------------------------------------+-------------------------------------------+
| Prisma ORM 7 + Driver Adapter
+-----------------------------------------v-------------------------------------------+
| PERSISTENCE LAYER (MySQL 8.0 Relational Database via Connection Pooling) |
| - Tables: User, Outlet, Order, OrderItem, Payment, OutletMenuItem Overrides, Audit |
+-------------------------------------------------------------------------------------+
```

3. Database Design & Core Entity Relationships

The relational schema is defined in Prisma ORM 7 (`schema.prisma`), enforcing strict referential integrity, foreign key cascading, and optimized indexes:

Entity	Key Fields & Types	Referential Constraints & Business Logic
User	id (UUID), email (unique), phone (unique), passwordHash, fullName, role (enum), marketingConsent (bool)	RBAC roles: CUSTOMER, KITCHEN_STAFF, OUTLET_MANAGER, ADMIN. Argon2id hashed passwords. Staff and managers assigned to specific outlet.
Outlet	id (UUID), name, slug (unique), address, city, latitude, longitude, deliveryRadiusKm, minPrepMinutes, isActive	Defines geographic service radius, base kitchen operational floor (T_min), and delivery qualification thresholds.
Order	id (UUID), orderNumber (unique), customerId (FK), outletId (FK), status (enum), fulfillmentType (enum), subtotal, gstAmount, deliveryFee, totalAmount, deliveryAddressJson, placedAt, promisedTime	Enforces OrderStatus state machine. Stores binding promised preparation time and timestamps. Cascades delete on test cleanup.
OrderItem	id (UUID), orderId (FK), itemName, unitPriceSnapshot, quantity, lineTotal, selectedOptionsJson (Text)	Snapshots item name, options, and unit prices at purchase. Selected customizations stored as JSON for KDS rendering.
Payment	id (UUID), orderId (FK), method (enum: UPI/CARD), status (enum: PENDING/PAID/FAILED), amount, transactionRef, failureReason, paidAt	Transactional payment records. Unique idempotency check prevents duplicate payments and race conditions.

4. REST API Specification & Real-Time Socket.IO Protocol

All endpoints follow RESTful conventions under `/api/*`, utilizing structured JSON payloads, standard HTTP status codes, and unified error envelopes `{ success: false, error: string, code: string }`:

Endpoint	Method & Auth	Input / Parameters	Description & Security Rule
/api/auth/register	POST (Public)	RegisterInput schema	Registers customer with Argon2id hash; marketing consent defaults unchecked.
/api/auth/login	POST (Public)	emailOrPhone, password	Authenticates credentials; issues signed JWT with role and assignedOutletId claims.

/api/outlets	GET (Public)	None	Returns active restaurant branches with minimum prep floor and delivery radii.
/api/menu	GET (Public)	?outletId=...	Returns catalog with outlet-specific pricing and real-time 86ing availability.
/api/orders/checkout	POST (Customer)	CheckoutInput (Zod)	Verifies pricing, calculates 5% GST, evaluates AI prep time, creates PLACED order.
/api/orders/kitchen/queue	GET (Staff/Admin)	?outletId=...&status;=...	KDS queue. Staff restricted to assigned outlet (403 on CROSS_OUTLET). Admin supports 'ALL'.
/api/orders/:id/status	PATCH (Staff/Admin)	{ status: OrderStatus }	Advances order state machine. Rejects invalid skips and cross-outlet mutations.
/api/orders/:id/prep-time	PATCH (Manager/Admin)	{ promisedTime, reason }	Allows authorized managers to override promised time with an audit reason.
/api/payments	POST (Customer)	PaymentRequest (Zod)	Server-side mock payment engine with idempotency key deduplication.
/health	GET (Public)	None	System liveness probe returning `{ status: 'ok', service: 'dumline-server' }`.

Real-Time Socket.IO Multi-Room Architecture:

• **Channel Rooms:** Kitchen tablets join `outlet:${assignedOutletId}`. Central Admin joins `outlet:ALL`. Individual customers join `order:${orderId}`.
• **Dual-Cast Publishing:** Order mutations publish to both `outlet:${outletId}` and `outlet:ALL`, keeping branch tablets decoupled from centralized command.
• **Events:** `order:created`, `order:status-updated`, `order:overdue-alert`, `order:prep-time-overridden`, `kitchen:notify`.

5. Authentication & Role-Based Access Control (RBAC)

DUMLINE enforces fine-grained access control across 4 distinct user roles, ensuring zero privilege escalation and strict multi-tenant branch isolation:

Role	Access Scope & Routing	Permissions & Operational Capabilities	Security Boundaries Enforced
CUSTOMER	Storefront (`/`), Cart, Checkout, Order Tracker (`/orders`)	Guest menu browsing, single-outlet cart creation, checkout submission, live tracking.	Strictly forbidden from `/kds`, `/manager`, `/admin` and kitchen queue APIs (HTTP 403).
KITCHEN_STAFF	Kitchen Display System (`/kds`)	Station-locked KDS queue view, status transitions (`ACCEPTED` $\to$ `PREPARING` $\to$ `READY`).	Station locked to assigned branch; querying other outlets or 'ALL' returns `403 CROSS_OUTLET_FORBIDDEN`.
OUTLET_MANAGER	Manager Portal (`/manager`) & Branch KDS (`/kds`)	Prep-time manual overrides, localized stock toggles (86ing), branch analytics, cancellation approval.	Scoped to assigned branch; cannot alter items or orders in other branches.
ADMIN	Central Menu Admin (`/admin`) & Central KDS Command (`/kds`)	Global catalog governance, multi-branch 'All Kitchens' centralized KDS, branch creation, cross-outlet metrics.	Unrestricted access across all branches; prominent 'All Kitchens' desktop navigation item.

6. Customer Ordering Workflow & Financial Integrity

• **Single-Outlet Cart Isolation:** Each cart is strictly bound to one outlet ID in Zustand storage. If a customer attempts to select items from another branch, an interactive confirmation modal requires explicit approval before flushing the existing cart, preventing cross-branch delivery impossibilities.
• **Mandatory Authentication Gate:** Guests can freely browse dishes and customize spice profiles, but checkout requires authentication. Guest checkouts trigger an inline modal to log in or register before order commitment.
• **GST Pricing Compliance:** All listed retail prices are 5% GST-inclusive as required by Indian restaurant tax laws. The checkout engine breaks down `Base = Retail / 1.05` and `GST = Retail - Base` without double-charging.
• **Simulated Direct Delivery:** Addresses are verified against the selected outlet's configurable delivery radius ($r \le 7.0\text{ km}$) with simulated tiered delivery fees.

7. Branch-Specific KDS and Central Admin All-Kitchens Workflow

The Kitchen Display System provides two distinct operational modes tailored to user responsibilities:

• **Branch KDS (Kitchen Staff):** Staff assigned to Kondapur, Gachibowli, or Kukatpally are locked to their kitchen. The header displays ` {Branch Name} (Station Locked)`. Backend authorization strictly enforces that staff can neither query other queues nor update unauthorized orders (`403 CROSS_OUTLET_FORBIDDEN`).
• **Centralized All-Kitchens KDS (Central Admin):** Central Admin is granted multi-branch oversight. The dashboard defaults to `outletId=ALL`, aggregating active orders from all branches without duplicates. An Executive Branch Summary Strip displays active and delayed counts per branch with 1-click filtering.
• **Status Tabs & Overdue Filtering:** Filters include `ALL ACTIVE`, `NEW (PLACED)`, `ACCEPTED`, `PREPARING`, `READY`, `COMPLETED`, and `DELAYED (OVERDUE)`. Overdue orders display an urgent pulsing indicator and audible chime.

8. Menu CMS, Customization Engine & Inventory 86ing

• **Authentic Culinary Taxonomy:** Master catalog covers Royal Hyderabadi Dum Biryanis, Starters, Kebabs, Salans, Desserts, and Irani Chai.
• **Strict Category Customization Isolation:** Customization options are strictly bound to culinary categories: Biryani options offer Mirchi Ka Salan choice, spice profile (Mild, Nizam Royal, Mirch Masala), and raita; Chai options offer sweetness and ginger level; Desserts offer portion size. Non-biryani items never display inappropriate biryani spice options.
• **Outlet-Level 86ing (Stock Depletion):** Outlet Managers can toggle availability for any dish at their outlet (`PATCH /api/menu/availability`). 86ing an item at Kondapur disables cart addition for Kondapur while remaining fully available at Kukatpally and Gachibowli.

9. Server-Side Mock Payment Contract & Idempotency

To provide a realistic assessment simulation without exposing real financial credentials, DUMLINE implements a server-side idempotent payment engine supporting UPI and Card payment methods:

Payment Method	Success Test Trigger	Failure Test Trigger	Idempotency & Settlement Behavior
UPI (Unified Payments)	VPA: **success@upi** Status: `PAID`	VPA: **fail@upi** Status: `FAILED` (INSUFFICIENT_FUNDS)	Transactions require client UUID idempotency key. Duplicate submissions return cached transaction results without re-executing.
Credit / Debit Card	Card ending: **4242** Status: `PAID`	Card ending: **0002** Status: `FAILED` (CARD_DECLINED)	Zero real cardholder data persisted. Validates expiry date and CVV format prior to mock settlement.

10. AI Preparation-Time Estimator & Deterministic Fallback

To balance intelligent kitchen pacing with zero-failure operational resilience, DUMLINE implements a four-tier preparation formula:

1. **AI Estimate (E_{AI}):** Evaluated via Google Gemini 1.5 Flash using official SDK with structured JSON schema output (`{ estimatedMinutes, reasoning, confidence }`). Factored inputs: item categories, complexity tags (`DUM_COOKING`, `TANDOOR`, `READY_TO_SERVE`), active kitchen orders, and peak dining hours.

2. **Parallel Kitchen Station Safeguard:** The AI prompt and scheduling logic model concurrent preparation across stations (Biryani Station and Dessert/Salan Station prepare simultaneously), eliminating additive multiplication of cook times.
3. **Deterministic Fallback (E_{Fallback}):** If Gemini API is unreachable, times out (>3000ms), or returns an invalid schema, the system executes an instant mathematical calculation:
   E_Fallback = BasePrep (10m) + max(StationMaxPrep) + (ActiveOrders × 2m) + (AdditionalPortions × 1.5m)
4. **Binding Promised Preparation Time (T_{Promised}):** The legally binding customer promised time presented to KDS:
   T_Promised = max(E_Active, T_Min), clamped strictly within [5, 120] minutes.
5. **Automated Overdue Alerting:** When Elapsed Time $\ge T_{\text{Promised}} + 10\text{ minutes}$ and status is not `READY`/`COMPLETED`, an overdue alert is published to KDS and customer screens with audio chimes and red pulsing visual tokens.

11. Mandatory Assessment Seed Fixtures

As mandated by the IsoCode Labs Software Engineering Intern Assessment brief, the database seed registers three immutable reference orders:

Order Number	Outlet Location	Status	Payment Details	Total Amount	Time Schedule
WEB-778120	Kondapur	**PREPARING**	UPI Paid (success@upi)	■1,180.00	Placed: 20:14 Promised: 20:59
WEB-778131	Gachibowli cloud kitchen	**READY**	Card Paid (ending 4242)	■640.00	Placed: 20:21 Promised: 21:01
WEB-778146	Kukatpally	**PREPARING (Overdue)**	UPI Paid (success@upi)	■1,050.00	Placed: 20:26 Promised: 21:16

12. Security Architecture & Threat Mitigation

• **Argon2id Hashing:** Passwords encrypted with industry-standard Argon2id algorithm, resistant to GPU/ASIC cracking.
• **JWT Token Defense:** Signed JSON Web Tokens with strict expiration (`7d`). Role and outlet claims verified on every request.
• **Input Sanitization with Zod:** All HTTP query parameters, path variables, and request bodies are validated against shared Zod schemas before hitting business services.
• **Rate Limiting:** In-memory token bucket rate limiting on sensitive authentication and payment endpoints preventing brute-force abuse.
• **Zero Credential Exposure:** Environment variables isolated via `.env`; no API keys or database secrets committed.

13. Comprehensive Testing Approach & Actual Test Results

DUMLINE features a multi-tiered automated verification suite executed against real MySQL instances and live API servers:

Test Suite File	Scope & Assertions Verified	Result
fixtures.test.ts	Mandatory IsoCode Labs seed orders (WEB-778120, WEB-778131, WEB-778146) verified exactly.	**PASS (3/3)**
auth.test.ts	Argon2id hashing, JWT payload preservation, registration duplicate prevention.	**PASS (3/3)**
cart-checkout.test.ts	GST 5% inclusive pricing, delivery radius validation, stock 86ing cart blocking.	**PASS (3/3)**
payment-idempotency.test.ts	UPI & Card mock triggers, idempotency key replay preventing duplicate charges.	**PASS (3/3)**
order-state-machine.test.ts	Strict sequential transitions (PLACED $\to$ ACCEPTED $\to$ PREPARING $\to$ READY $\to$ COMPLETED).	**PASS (3/3)**
ai-prep.test.ts	Gemini AI SDK structured parsing, deterministic fallback formula, parallel station bounds.	**PASS (3/3)**
customer-e2e.test.ts	Complete customer lifecycle: browsing $\to$ customization $\to$ checkout $\to$ live tracking.	**PASS (4/4)**

customer-to-kitchen-flow.test.ts	Customer order placement $\to$ real-time KDS dispatch $\to$ kitchen state updates.	**PASS (4/4)**
rbac-access.test.ts	Role boundaries: customer blocked from KDS; staff blocked from manager/admin endpoints.	**PASS (6/6)**
master-audit.test.ts	Comprehensive 20-point regression audit covering all requirements end-to-end.	**PASS (20/20)**
branch-kds.test.ts	Branch staff isolation (403 on cross-outlet), Central Admin 'All Kitchens' KDS command, overdue query.	**PASS (12/12)**

Compilation & Static Analysis Verification:
• **TypeScript Typecheck:** `npm run typecheck --workspaces` exited with **0 errors** across `@dumline/server`, `@dumline/web`, and `@dumline/shared`.
• **Next.js Production Build:** `npm run build --workspace=@dumline/web` successfully compiled all 11 static and dynamic routes.

14. Conclusion & Submission Sign-Off

DUMLINE delivers an uncompromising, production-ready implementation fulfilling every requirement of the IsoCode Labs Software Engineering Intern Assessment. By coupling an authentic royal visual identity with robust transactional integrity, resilient real-time WebSockets, intelligent dual-engine prep estimation, and strict branch-isolated KDS operations, the platform sets a benchmark for modern restaurant technology systems.

Verified Submission Repository: https://github.com/Srionline610/DUMLINE